

บทที่ 3

ไคเร็กซ์เอ็กซ์ กราฟฟิกส์

3.1 ความรู้เบื้องต้นกับไคเร็กซ์เอ็กซ์ กราฟฟิกส์

ไคเร็กซ์เอ็กซ์ กราฟฟิกส์ (DirectX Graphics) เป็นคอมโพเนนต์ใหม่ ในไคเร็กซ์เอ็กซ์ เวอร์ชัน 8 ซึ่งรวมเอาการทำงานของ คอมโพเนนต์เดิม คือ ไคเร็กซ์ดรอว์ (DirectDraw) กับ ไคเร็กซ์ทรีดี (Direct3D) มาเป็นอินเตอร์เฟซเดียวคือ ไคเร็กซ์ทรีดี ซึ่งจะรวมการทำงานในการแสดงผลภาพ 2 มิติ และ 3 มิติ ไว้ในอินเตอร์เฟซนี้

ไคเร็กซ์ทรีดี ได้รับการออกแบบ เพื่อช่วยในการสร้างเกมที่แสดงผลภาพแบบ 2 และ 3 มิติ บนเครื่องคอมพิวเตอร์ภายใต้ระบบปฏิบัติการวินโดวส์ ซึ่งกล่าวได้ว่า ไคเร็กซ์ทรีดี คือซอฟต์แวร์อินเตอร์เฟซ ซึ่งจัดการการแสดงผลที่รองรับการเข้าถึงอุปกรณ์แสดงผล โดยยังคงรักษาความเข้ากันได้กับกราฟฟิกส์ ไวซ์อินเตอร์เฟซ (Graphics Device Interface : GDI) และจัดการให้มีการใช้งานในรูปแบบที่ไม่ขึ้นกับชนิดของอุปกรณ์ (Device Independent) และมีความสามารถในการใช้คุณสมบัติพิเศษที่อยู่ในฮาร์ดแวร์ได้

ไคเร็กซ์ทรีดี นี้เป็นแอปพลิเคชันโปรแกรมมิ่งอินเตอร์เฟซระดับล่าง ซึ่งเป็นเครื่องมือสำหรับโปรแกรมเมอร์ที่ต้องการสร้างเกมหรือโปรแกรมมัลติมีเดียที่แสดงผล 3 มิติ ที่มีประสิทธิภาพสูง บนระบบปฏิบัติการวินโดวส์ ซึ่งการใช้เอพีไอไคเร็กซ์ทรีดีนี้ ทำให้โปรแกรมสามารถติดต่อโดยตรงกับฮาร์ดแวร์เร่งความเร็วในระดับล่าง จึงมีความยืดหยุ่นในการใช้งานสูง

3.2 คุณสมบัติของไคเร็กซ์ทรีดี

- 3.2.1 สนับสนุนการใช้งาน Depth Buffer (โดยใช้ z-buffer หรือ w-buffer)
- 3.2.2 สามารถเรนเดอร์ภาพได้ทั้งในแบบ Flat Shading และ Gouraud Shading
- 3.2.3 สามารถสร้างแหล่งกำหนดแสงได้หลายชนิดและหลายแหล่ง
- 3.2.4 รองรับการใช้วัสดุแมตทีเรียล (material) และพื้นผิวเท็กซ์เจอร์ (texture) รวมไปถึงการทำ mipmapping (mipmap)
- 3.2.5 สนับสนุนซอฟต์แวร์จำลองการใช้งานของฮาร์ดแวร์ (Software Emulation Driver) ที่มีประสิทธิภาพสูง
- 3.2.6 มีการทำงานโดยไม่ขึ้นกับชนิดของฮาร์ดแวร์
- 3.2.7 รองรับการแปลงพิกัดวัตถุ (Transformation) และการขริบภาพ (Clipping)
- 3.2.8 สนับสนุนคำสั่งพิเศษที่มีอยู่ในซีพียู สามารถรองรับสถาปัตยกรรมแบบ Intel MMX, Intel Streaming Single-Instruction, Multiple-Data (SIMD) Extensions (SSE) และสถาปัตยกรรมทรีดีของ AMD (AMD's 3D Architecture)
- 3.2.9 สนับสนุน Hardware Abstraction Layer (HAL)

3.2.10 สนับสนุนการทำ page flipping ด้วย back buffer จำนวนมากในโปรแกรมที่แสดงผลแบบเต็มจอภาพ (fullscreen)

3.2.11 สนับสนุนการตัด (Clipping) ในโปรแกรมประยุกต์ที่ทำงานในวินโดว์โหมด และแบบโหมดเต็มจอภาพ

3.2.12 รองรับการทำ 3D - z buffer

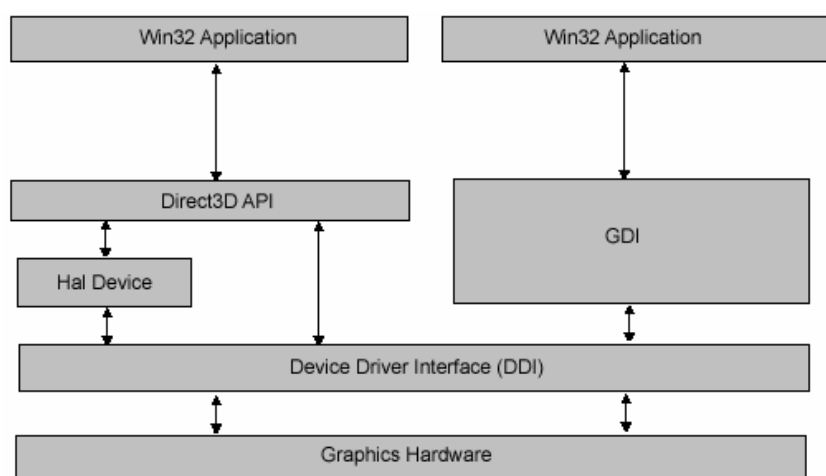
3.2.13 สามารถใช้งาน image-stretching hardware ได้

3.2.14 สามารถเข้าถึงหน่วยความจำของอุปกรณ์แสดงผล (display-device memory) แบบ Standard และ Enhance ได้ในขณะเดียวกัน

นอกจากนี้ยังมีคุณสมบัติอื่นๆ เช่น การใช้งานฮาร์ดแวร์แต่เพียงผู้เดียว (exclusive hardware access) และการเปลี่ยนความละเอียดในการแสดงผล (resolution switching)

ด้วยคุณสมบัติข้างต้นเหล่านี้ การพัฒนาโปรแกรมด้วยไคลเร็กซ์ตรีดี จึงมีประสิทธิภาพสูงกว่าการใช้กราฟฟิกส์ไวนเตอร์เฟส (GDI) ของวินโดว์และการเขียนโปรแกรมบนระบบปฏิบัติการดอส

ตามแผนผังข้างล่างนี้ แสดงถึงความสัมพันธ์ระหว่างอินเตอร์เฟสไคลเร็กซ์ตรีดี, กราฟฟิกส์ไวนเตอร์เฟส (Graphics Device Interface : GDI), ฮาร์ดแวร์แอบสแตรกชันเลเยอร์ (Hardware abstraction Layer : HAL) และ ฮาร์ดแวร์

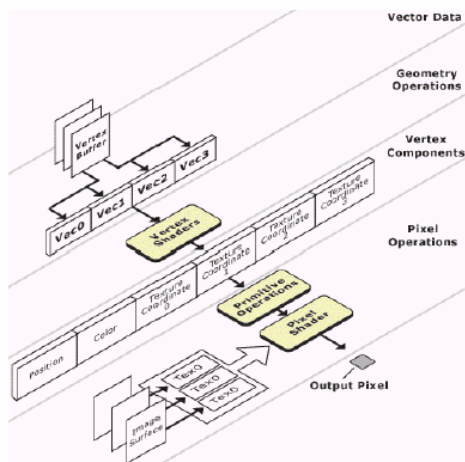


รูปที่ 3-1 แสดงความสัมพันธ์ระหว่างไคลเร็กซ์ตรีดีกับระบบคอมพิวเตอร์

เมื่อนำมาเปรียบเทียบระหว่างไคลเร็กซ์ตรีดี กับ กราฟฟิกส์ไวนเตอร์เฟส (GDI) ทั้งคู่จะมีการเข้าถึงกราฟฟิกส์ฮาร์ดแวร์ (Graphics Hardware) ผ่านไวนเตอร์เฟสไดรเวอร์ (Device Driver Interface : DDI) สิ่งที่แตกต่างกันไป คือ ไคลเร็กซ์ตรีดีมีข้อดีกว่าของหน้าที่การทำงานกับฮาร์ดแวร์เมื่อมีการเลือกใช้ HAL ซึ่งไวนเตอร์เฟส HAL นี้จัดการความเร็วทางด้านฮาร์ดแวร์ บนพื้นฐานของชุดการทำงานที่สนับสนุนกราฟฟิกส์

หากฮาร์ดแวร์ที่มีอยู่ในเครื่องคอมพิวเตอร์ใดไม่สนับสนุนการเร่งความเร็วในส่วนใดของไปป์ไลน์ก็ตาม ไคลเร็กซ์ตรีดีจะใช้ซอฟต์แวร์ที่มีประสิทธิภาพสูงในการทำงานในส่วนนั้นแทน แต่การทำงานของซอฟต์แวร์ ก็ยังมีประสิทธิภาพต่ำกว่าใช้ฮาร์ดแวร์

3.3 สถาปัตยกรรมของไดเรกซ์ทรีดี (Architectural overview of Direct3D)



รูปที่ 3-2 โครงสร้างการทำงานของไดเรกซ์กราฟฟิก

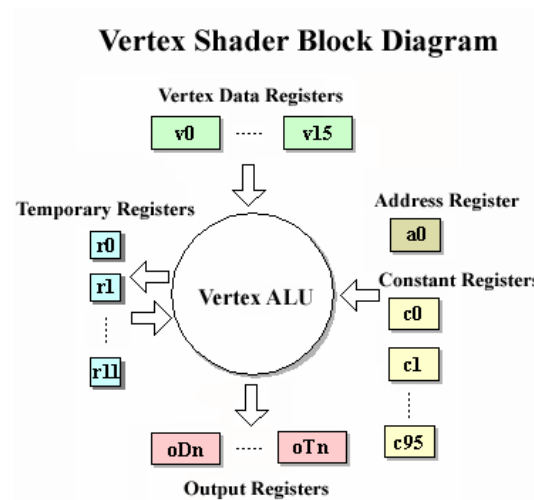
ในไดเรกซ์ทรีดีมี 2 ส่วน ที่สามารถโปรแกรมได้คือ เวอร์เท็กซ์เชดเดอร์ (Vertex shader) และ พิกเซลเชดเดอร์ (pixel shader)

จากรูปด้านบน การทำงานของเวอร์เท็กซ์เชดเดอร์ จะรับข้อมูลจากบัฟเฟอร์ (vertex buffer) ที่เก็บข้อมูลเวอร์เท็กซ์ และนำมาประมวลผล ซึ่งสามารถกำหนดกระบวนการที่จะทำได้โดยใช้ ไดเรกซ์เอ็กซ์ เวอร์เท็กซ์ เชดเดอร์ แอสเซมเบลอร์ (DirectX vertex shader assembler)

จากนั้น จะต้องเรียกฟังก์ชันในการวาดข้อมูล เช่น DrawPrimitive เป็นต้น ซึ่งเมื่อเรียกฟังก์ชันเหล่านี้แล้ว จุดแต่ละจุดที่ได้จากฟังก์ชันก็จะส่งเข้าสู่พิกเซลเชดเดอร์ ข้อมูลที่ส่งให้กับพิกเซลเชดเดอร์ประกอบด้วยตำแหน่งบนจอภาพ, สี, เท็กซ์เจอร์ และ คู่อันดับแสดงตำแหน่งของเท็กซ์เจอร์ พิกเซลเชดเดอร์จะประมวลผลตามคำสั่งที่กำหนดโดยใช้ไดเรกซ์เอ็กซ์ พิกเซล แอสเซมเบลอร์ (DirectX pixel assembler)

นอกจากข้อมูลที่ได้รับจากฟังก์ชันในการวาดรูป เช่น DrawPrimitive แล้ว พิกเซลเชดเดอร์ยังต้องการข้อมูลของเท็กซ์เจอร์ในรูปแบบของเท็กซ์เจอร์เซอร์เฟส (texture surface) อีกด้วย ซึ่งจะให้ร่วมกับข้อมูลคู่อันดับแสดงตำแหน่งของเท็กซ์เจอร์ ในการกำหนดสีที่จะปรากฏ แล้วจึงส่งค่าพิกเซล (pixel) ไปยังเฟรมบัฟเฟอร์ (frame buffer) เพื่อแสดงผลออกจจอภาพต่อไป

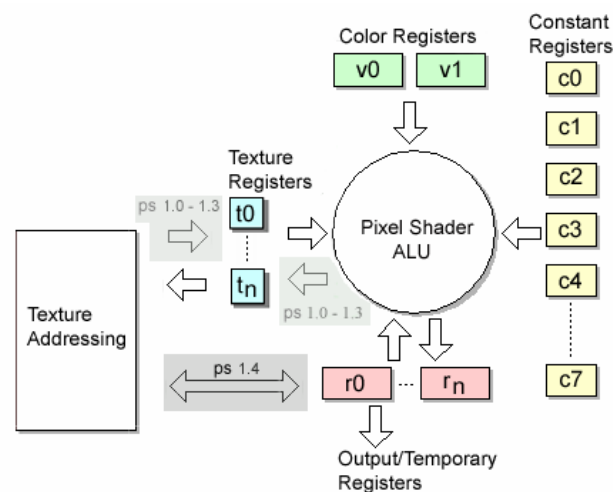
3.3.1 สถาปัตยกรรมเวิร์ทเท็กซ์เชเดอร์



รูปที่ 3-3 แสดงแผนผังสถาปัตยกรรมเวิร์ทเท็กซ์เชเดอร์

แผนผังรูปที่ 3-3 ข้างบนอธิบายสถาปัตยกรรมเวิร์ทเท็กซ์เชเดอร์ โดยส่งข้อมูลแบบเวิร์ทเท็กซ์ คาค่าสตรีม (vertex data stream) ไปยังเชเดอร์จากกราฟฟิกไปป์ไลน์ มีการกระทำคำสั่งบนข้อมูลโดยใช้ อาริธเมติก ลอจิก ยูนิต (Arithmetic Logic Unit : ALU) และส่งออกไปแปลงเวิร์ทเท็กซ์คาค่าไปยัง กราฟฟิกไปป์ไลน์สำหรับการทำงานอื่นๆ

3.3.2 สถาปัตยกรรมพิกเซลเชเดอร์



รูปที่ 3-4 แสดงแผนผังสถาปัตยกรรมพิกเซลเชเดอร์

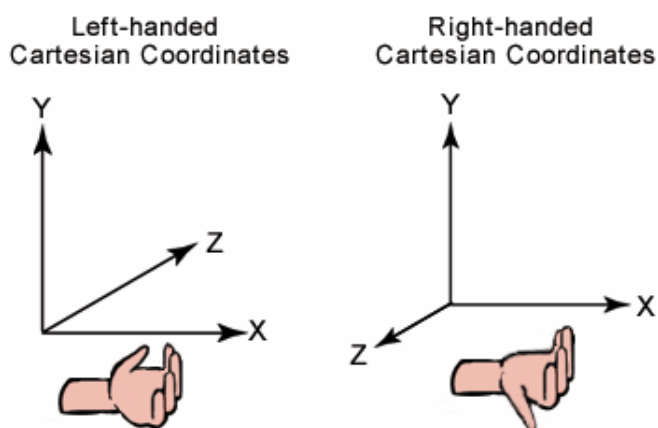
แผนผังรูปที่ 3-4 แสดงถึงสถาปัตยกรรมพิกเซลเชเดอร์ สถาปัตยกรรมนี้ใช้พิกเซลคาค่า โดย ข้อมูลนั้นเป็นคลิพ สเปซ เวิร์ทเท็กซ์ (Clip space vertex) ค่าของสีดิฟฟิวส์ (diffuse) และสเปกคิวลาร์

(specular) จากค่าสีในรีจิสเตอร์สี (color register) v0 และ v1 ตำแหน่งของเท็กซ์เจอร์และเท็กซ์เจอร์ที่ใช้มาจากค่าในรีจิสเตอร์เท็กซ์เจอร์ (texture register) t0, ... tn จากข้อมูลในขั้นตอนการกำหนดเท็กซ์เจอร์ (texture setup stage) และข้อมูลในเวอร์เท็กซ์ กล่าวได้ว่า พิกเซลเชดเดอร์ทำงานหลังจากมีการประมวลผลข้อมูล ออกมาเป็นตำแหน่งของพิกเซลและค่าสีของพิกเซลนั้นๆ

3.4 ทฤษฎีระบบพิกัด 3 มิติ

3.4.1 ระบบพิกัด 3 มิติ (3-D Coordinate System)

แบบฉบับของโปรแกรมประยุกต์ที่แสดงผลกราฟฟิก 3 มิติ จะมี 2 แบบในการแสดงระบบพิกัด 3 มิติ คือ มือซ้าย และมือขวา ในระบบพิกัดทั้งคู่ แกนด้านบวกของ x จะไปทางด้านขวา และแกนด้านบวกของ y จะขึ้น ไปข้างบน สามารถสังเกตเพื่อจดจำได้จากทิศทางของแกนบวกของ z ได้ โดยการหงายมือขึ้น จากนั้นกางนิ้วมือออกชี้ไปยังแกนบวกของ x แล้วงอนิ้วขึ้นด้านบนตามทิศทางของแกนบวกของ y ทิศทางที่นิ้วโป้งชี้ไป จะเป็นทิศทางของแกนบวกของ z ตามมือที่ใช้



รูปที่ 3-5 แสดงระบบพิกัด 3 มิติ แบบมือซ้ายและมือขวา

ไมโครซอฟท์ ไคลเร็กซ์ตรีดี มักจะใช้ระบบพิกัด 3 มิติ แบบมือซ้าย ถ้าต้องการทำโปรแกรมที่เป็นระบบมือขวา จะต้องเปลี่ยนข้อมูลบางอย่าง ดังนี้

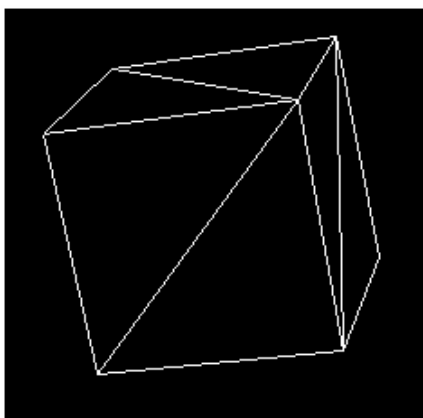
- เปลี่ยนรายการของจุดของสามเหลี่ยม (triangle vertices) โดยสลับแบบตามเข็มนาฬิกาจากด้านหน้า ตัวอย่างเช่น ถ้าลำดับจุดเป็น v0, v1, v2 ให้เปลี่ยนข้อมูลก่อนที่จะส่งค่าไปให้ Direct3D คือ v0,v2, v1
- ใช้การวิวแมทริกซ์เพื่อปรับพิกัดเวิลด์สเปซ (World Space) ด้วย -1 ในทิศทางของแกน z ซึ่งทำได้โดย สลับเครื่องหมายของ _31, _32, _33, _34 แถวที่สามในแมทริกซ์ ของโครงสร้าง D3DMATRIX ที่ใช้สำหรับวิวแมทริกซ์

สำหรับฟังก์ชันที่จะใช้บอกไคลเร็กซ์ตรีดีว่า เราจะใช้ระบบพิกัด 3 มิติ แบบไหนนั้น ก็คือ D3DXMatrixLookAtLH() สำหรับระบบมือซ้าย และ D3DXMatrixLookAtRH() สำหรับระบบมือขวา

3.4.2 ชุดของจุดของวัตถุ 3 มิติ (3-D Primitive)

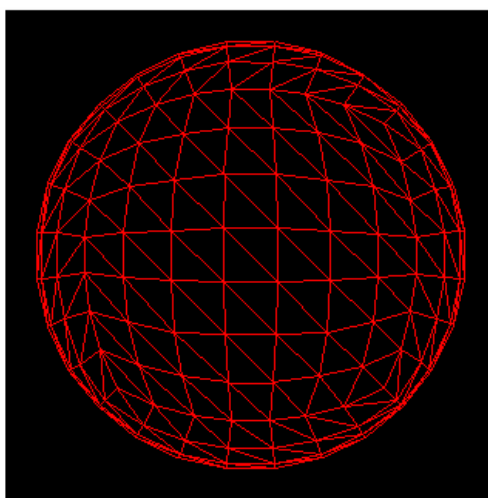
3-D Primitive เป็นชุดของจุด (vertices) ที่เป็นของวัตถุ 3 มิติ และพริมีทีฟ ที่ง่ายที่สุด คือ ชุดของจุดในระบบพิกัด 3 มิติ ที่ถูกเรียกว่า รายการจุด (Point List)

โดยปกติ 3-D Primitive จะหมายถึงโพลีกอน เป็นรูปปิด 3 มิติ ที่มีจุดที่มีการเชื่อมกันอย่างน้อย 3 จุด โพลีกอนแบบพื้นฐานที่สุดคือรูปสามเหลี่ยม ไมโครซอฟท์ ไคเร็กซ์ทรีดี ส่วนใหญ่ใช้สามเหลี่ยมในการประกอบโพลีกอน เพราะจุดสามจุดในรูปสามเหลี่ยมนั้นจะรับประกันได้ว่าอยู่ในระนาบเดียวกัน เพื่อให้การเรนเดอร์นั้นเป็นไปอย่างมีประสิทธิภาพ จึงสามารถประกอบรูปสามเหลี่ยมเป็นโพลีกอนหรือวัตถุที่มีความซับซ้อนได้



รูปที่ 3-6 แสดงรูปลูกบาศก์ที่ประกอบกันขึ้นมาจากรูปสามเหลี่ยม

รูปข้างบนนี้แสดงรูปสี่เหลี่ยมลูกบาศก์ ในแต่ละด้านประกอบขึ้นมาจากสามเหลี่ยม 2 รูป ซึ่งรูปสี่เหลี่ยมลูกบาศก์นี้ประกอบขึ้นมาจากชุดของรูปสามเหลี่ยม นอกจากนี้ ยังสามารถลงเท็กซ์เจอร์และชนิดวัสดุ(material) บนพื้นผิววัตถุได้ เพื่อสร้างพื้นผิวของพริมีทีฟ ให้เห็นเป็นลูกสี่เหลี่ยมลูกบาศก์ทรงตันได้



รูปที่ 3-7 แสดงภาพทรงกลมที่ประกอบขึ้นมาจากรูปสามเหลี่ยม

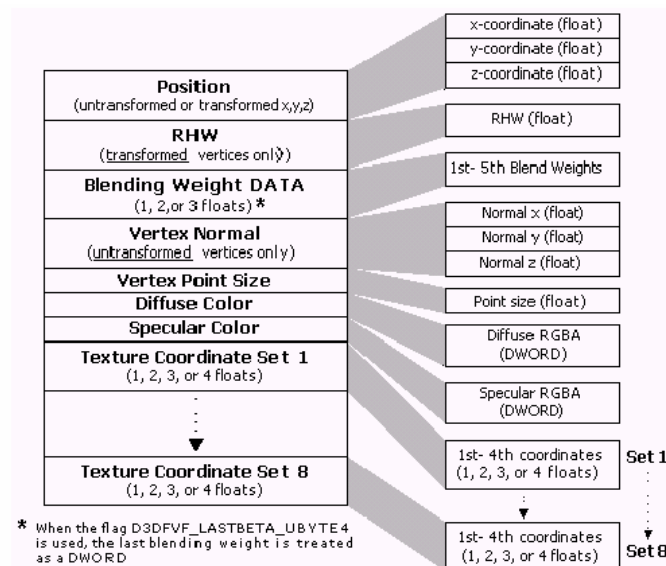
จากรูปด้านบนนี้ แสดงถึงทรงกลมที่ประกอบขึ้นมาจากสามเหลี่ยม ซึ่งแสดงให้เห็นว่าสามารถใช้สามเหลี่ยมเพื่อสร้างพริมีทีฟสำหรับรูปทรงที่มีพื้นผิวราบเรียบได้ เมื่อทำการลงพื้นผิวจะเห็นเป็นรูปทรงกลมที่พื้นผิวเรียบได้

3.4.3 เวกเตอร์ (Vector)

เวกเตอร์ในไคลเร็กซ์ตรีดี มีความหมายว่า ค่าที่แสดงถึงระยะและทิศทางของวัตถุ เช่น เวกเตอร์ที่แสดงพิกเซลบนจอภาพ จะประกอบด้วยค่าในแกน x และ y ถ้าทิศทางในแนวแกน x เป็นบวก จะหมายถึงพิกเซล นั้นจะอยู่ไปทางด้านขวาของจอภาพ ถ้าทิศทางในแนวแกน y เป็นบวก จะหมายถึงพิกเซลที่ตำแหน่ง y นั้นจะอยู่ไปทางด้านล่างของจอภาพ เพราะตำแหน่งเริ่มต้นของจอภาพ คือมุมบนซ้ายสุด (0, 0) ซึ่งเวกเตอร์ที่ประกอบด้วยค่า 2 ค่าอย่างนี้เราเรียกว่าเวกเตอร์ 2 มิติ (2-D Vector) ซึ่งในไคลเร็กซ์ตรีดี จะใช้เวกเตอร์อยู่ 3 ประเภท คือ เวกเตอร์ 2 มิติ, เวกเตอร์ 3 มิติ, เวกเตอร์ 4 มิติ

3.4.4 เวกอร์เท็กซ์ (Vertex)

เวกอร์เท็กซ์ คือจุดในโลก 3 มิติ โดยอย่างน้อยจะมีเวกเตอร์ 1 เวกเตอร์ การสร้างเวกอร์เท็กซ์ในไคลเร็กซ์ตรีดี จะใช้วิธีสร้างแบบฟเล็กซิเบิลเวกเตอร์ (Flexible Vector Format : FVF) ซึ่งเป็นโครงสร้างที่ประกอบไปด้วยเวกเตอร์ที่ต้องการจะใช้งานเท่านั้น เวกเตอร์ไหนที่ไม่ใช้ก็ไม่จำเป็นต้องใส่ โดยจะต้องเรียงลำดับเวกเตอร์และใช้ประเภทของตัวแปรตามรูปแบบในรูป 3-8 นี้



รูปที่ 3-8 รูปแบบลำดับเวกเตอร์ในเวกอร์เท็กซ์

3.4.5 โมเดลสเปซ (Model Space)

โมเดลสเปซ (Model Space) เป็นพิกัดในโลก 3 มิติ (3-D Coordinate) ที่ใช้ภายในโมเดลวัตถุแต่ละอัน ซึ่งมีขอบเขตเป็นสี่เหลี่ยม โดยโมเดลจะประกอบขึ้นมาจากสามเหลี่ยมหลายๆ ชิ้น ซึ่งรูป

สามเหลี่ยมแต่ละชิ้นนั้นเกิดขึ้นจากเวอร์เท็กซ์ จำนวน 3 จุด โดยแต่ละจุดจะมีจุดกำเนิดอ้างอิงจุดเดียวกัน ซึ่งจุดกำเนิดนี้จะอยู่ที่กลางโมเดลแต่ละอัน

สำหรับพิกัดต่างๆ ที่โมเดลแต่ละอันใช้นี้จะเรียกว่าพิกัดของตนเอง (Local Coordinate) ขอบเขตของโมเดลสเปซ ขึ้นอยู่กับขนาดของโมเดล ว่าจะมีความกว้าง, ยาว และสูงเพียงใด

3.4.6 รูปแบบในการให้แสงเงา (Shading Mode)

รูปแบบในการให้แสงเงา ใช้สำหรับการเรนเดอร์โพลีกอนให้มีผลกระทบจากแสงอย่างทั่วถึงปรากฏขึ้น ซึ่งรูปแบบในการให้แสงเงา จะกำหนดจากความหนาแน่นของสีและแสงที่จุดใดๆ บน พื้นผิวของโพลีกอน สำหรับโมโครซอฟท์ ไคเร็กซ์ทรีดีสนับสนุนการให้แสงเงา 2 รูปแบบ ได้แก่

- การให้แสงเงาแบบแบนราบ (Flat Shading) ในรูปแบบการให้แสงเงาแบบแบนราบ ไคเร็กซ์ทรีดี จะทำการเรนเดอร์โพลีกอนโดยใช้สีของวัสดุ(material)ของโพลีกอน ที่จุด (vertex) แรก ให้กับโพลีกอนทั้งหมด วัตถุ 3 มิติ ที่มีการเรนเดอร์ด้วยรูปแบบการให้แสงเงาแบบแบนราบ จะมีขอบอย่างชัดเจนระหว่างโพลีกอน ถ้าหากว่าเป็นคนละครนะบ ดังรูปที่ 3-8 เป็นการให้แสงเงาแบบแบนราบให้กับกาน้ำชา ซึ่งจะเห็นได้ว่าจะเห็นขอบของแต่ละโพลีกอนอย่างชัดเจน แต่การให้แสงเงาแบบนี้ จะมีข้อดีคือใช้เวลาในการคำนวณผลน้อยสุด



รูปที่ 3-9 แสดงการให้แสงเงาแบบแบนราบให้กับกาน้ำชา

- รูปแบบการให้แสงเงาแบบเกราด์ (Gouraud Shading)

เมื่อไคเร็กซ์ทรีดีเรนเดอร์โพลีกอนโดยใช้รูปแบบการให้แสงเงาแบบเกราด์ จะคำนวณสีสำหรับแต่ละจุดโดยใช้จุดปกติและค่าตัวแปรของแสง จากนั้นแทรกสีไปผ่านผิวหน้าของโพลีกอน การแทรกนี้จะทำแบบเป็นเชิงเส้น ตัวอย่างเช่น ถ้ามีส่วนประกอบที่เป็นสีแดงของสีในจุดแรก เท่ากับ 0.8 และส่วนประกอบที่เป็นสีแดงของจุด 2 เท่ากับ 0.4 ใช้การให้แสงเงาแบบเกราด์และโครงสร้างสี RGB จะกำหนดค่าส่วนประกอบสีแดงได้เท่ากับ 0.6 ให้กับจุด ที่ตำแหน่งกลางของเส้นระหว่างจุด (vertice) ทำให้สีเรียบเนียนกว่าการให้แสงเงาแบบแบนราบ

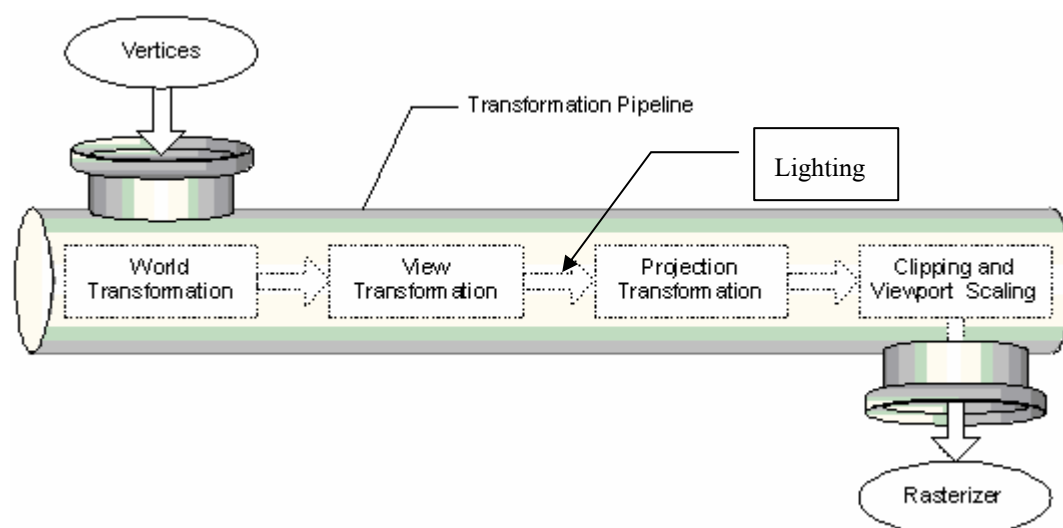


รูปที่ 3-10 แสดงการให้แสงเงาแบบเกราด์ให้กับกาน้ำชา

3.5 กระบวนการเรนเดอร์อ็อบเจกต์ 3 มิติ

ในการพัฒนาโปรแกรมประยุกต์ที่ใช้ไคเร็กซ์ตรีดนั้น จะใช้พื้นฐานอยู่บนทฤษฎีเกี่ยวกับ 3 มิติ ได้แก่ เวอร์เท็กซ์, โพลีกอนและคำสั่งที่ใช้ควบคุมข้อมูลเหล่านี้ รวมทั้งการเข้าถึงการแปลงพิกัดของวัตถุ ซึ่งเป็นความสามารถในการเข้าถึงไปป์ไลน์ของกราฟฟิก 3 มิติ ในส่วนของการเปลี่ยนแปลงตำแหน่ง (transformation), การให้แสง (Lighting) และขั้นตอนการแสดงผลเพื่อสร้างภาพ (Rasterization)

ซึ่งขั้นตอนในกระบวนการเรนเดอร์อ็อบเจกต์ 3 มิติ แบ่งเป็น 2 ขั้นตอน คือ กระบวนการแปรพิกัดและให้แสง (Transformation and Lighting : T&L) ซึ่งเป็นกระบวนการแปรพิกัดของเวอร์เท็กซ์ที่มีรูปแบบทศนิยม ไปสู่พิกัดแบบพิกเซลตามมุมมองของกล้องจำลองภายในซีน รวมถึงกระบวนการให้แสง ขั้นตอนที่ 2 คือ กระบวนการแปรมาสู่พิกเซล (Rasterization) เป็นขั้นตอนลงจุด เส้น และ รูปสามเหลี่ยม ด้วยภาพกราฟฟิกที่ได้จากกระบวนการ T&L

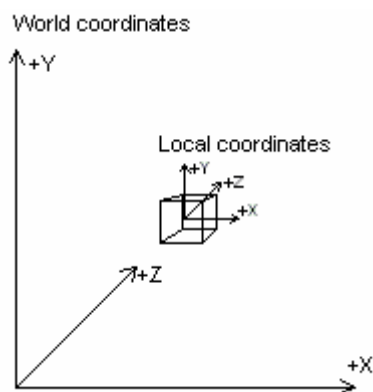


รูปที่ 3-11 ขั้นตอนในเรนเดอร์ไปป์ไลน์

3.5.1 กระบวนการแปรพิกัดและให้แสง (Transformation and Lighting : T&L)

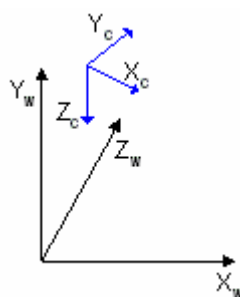
กระบวนการแปรพิกัด (Transformation) จะมีส่วนที่ทำหน้าที่ส่งอินพุตต่างๆ ผ่านไปยังฟังก์ชันการทำงานในรูปแบบไปป์ไลน์ของไคเร็กซ์ตรีด เรียกว่า Transformation Engine ซึ่งมีขั้นตอนการคำนวณเพื่อแปลงพิกัดต่างๆ ดังนี้

- World Transformation เป็นการคำนวณเพื่อแปลงพิกัด (Coordinate) หรือตำแหน่งของเวอร์เท็กซ์ (Vertex) ต่างๆ ที่ประกอบขึ้นเป็นโมเดลที่อยู่ในโมเดลสเปซ (Model Space) ให้เป็นพิกัดในเวิร์ลสเปซ (World Space) กล่าวได้ว่าเป็นการแปลงตำแหน่งเวอร์เท็กซ์เดิมที่มีความสัมพันธ์อยู่ในท้องถิ่นตนเอง (Local Coordinate) ให้มีความสัมพันธ์อ้างอิงกับตำแหน่งเวอร์เท็กซ์อื่นๆ ที่อยู่ในโลก 3 มิติ ด้วยกัน



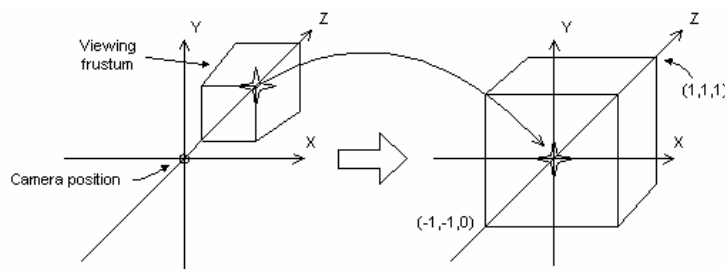
รูปที่ 3-12 แสดงความสัมพันธ์ระหว่างโมเดลในโมเดลสเปซที่มาอ้างอิงตำแหน่งใหม่ในเวิร์ลสเปซ

- View Transformation เป็นการคำนวณเพื่อแปลง ตำแหน่งของเวอร์เท็กซ์ต่างๆ บางส่วนของเวิร์ลสเปซ (World Space) ให้ไปเป็นพิกัด ในคามเร่าสเปซ (Camera Space) ซึ่งถือได้ว่าเป็นการแปลงให้มาอยู่ในมุมมองของผู้ดู



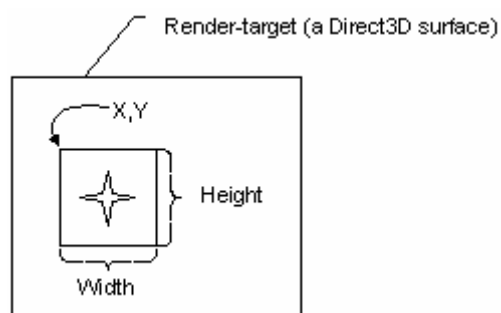
รูปที่ 3-13 แสดงความสัมพันธ์ระหว่างโมเดลในเวิร์ลสเปซกับมุมมองของกล้อง

- Lighting เป็นการคำนวณค่าสีต่างๆ ให้กับเวอร์เท็กซ์ เช่น สีที่ได้จากเทกเซล (Texel) ของเท็กซ์เจอร์ (Texture) ที่นำมาลงให้ตรง, สีของจุดเวอร์เท็กซ์นั้นๆ หรือสีจากแหล่งกำเนิดแสงต่างๆ เป็นต้น
- Projection Transformation เป็นการคำนวณเพื่อแปลงสิ่งที่อยู่ในคามเร่าสเปซ (Camera Space) ให้ไปอยู่ในโปรเจกชันสเปซ (Projection Space) โดยที่ขั้นตอนนี้จะมีการคำนวณตำแหน่งและขนาดของโมเดลให้ผิดแปลกไปจากเดิม เพื่อที่ว่าเมื่อถูกแปลงให้เห็นเป็นภาพ 2 มิติแล้ว จะได้เห็นเป็นแบบเพอร์สเปกทิฟ (Perspective) คือสิ่งที่อยู่ใกล้ตา จะใหญ่กว่าสิ่งที่อยู่ไกลออกไป ซึ่งจะทำให้มองเห็นเสมือนว่า ภาพ 2 มิตินั้นดูมีความลึก ซึ่งการเขียนโปรแกรมเพื่อทำงานในส่วน Transformation ทั้ง 3 แบบที่กล่าวมานี้จะต้องใช้เมทริกซ์ (Matrix)



รูปที่ 3-14 แสดงการทำโปรเจกชัน ซึ่งจะเห็นโมเดลในมุมมองภาพ 2 มิติที่มีความลึก

● Clipping and Viewport Scaling เป็นการทำงานที่แทรกก่อนจะไปถึงการทำงานในส่วนของการลงสี (Rasterization) นั่นคือ การตัดส่วนของโมเดลที่ถูกโมเดลอื่นบัง หรือตัดส่วนที่อยู่ด้านหลังออกไป แล้วก็ลดหรือขยายขนาดโมเดลต่างๆ ให้พอดีกับขนาดของความละเอียด (Resolution) ของหน้าจอที่ใช้ เป็นการช่วยลดการทำงานของขั้นตอนการลงสี คือไม่ต้องไปสนใจสิ่งที่มองไม่เห็นนั่นเอง



รูปที่ 3-15 หลังจากถูกวิวพอร์ตและขริบออก

3.5.2 กระบวนการลงสี (Rasterization)

ขั้นสุดท้ายในกระบวนการเรนเดอร์ไปป์ไลน์ จะเป็นการลงสี (Rasterization) โดยตัวที่ทำหน้าที่นี้จะเรียกว่า Rasterizer ซึ่งจะแปลงสิ่งที่อยู่ในโปรเจกชันสเปซ (Projection Space) ให้ไปอยู่ในสกรีนสเปซ (Screen Space) คือแปลงพิกัดของเท็กซ์เซล (Texel) ต่างๆ ที่เป็นแบบ 3 มิติ ให้เป็น 2 มิติ เมื่อเทียบกับขนาดของจอภาพ หลังจากนั้นเราก็จะได้ภาพปรากฏอยู่ในแบ็คบัฟเฟอร์ (Back Buffer) เพื่อที่เราจะได้สั่งให้ไดเรกทรีดีไวด์ธออบเจกต์ (Direct3D Device Object : DDI) จัดการนำข้อมูลในแบ็คบัฟเฟอร์ (Back Buffer) นี้ไปสลับหรือก๊อปปี้เป็นฟรอนท์บัฟเฟอร์ (Front Buffer) เพื่อให้ภาพไปปรากฏให้เห็นบนจอภาพต่อไป